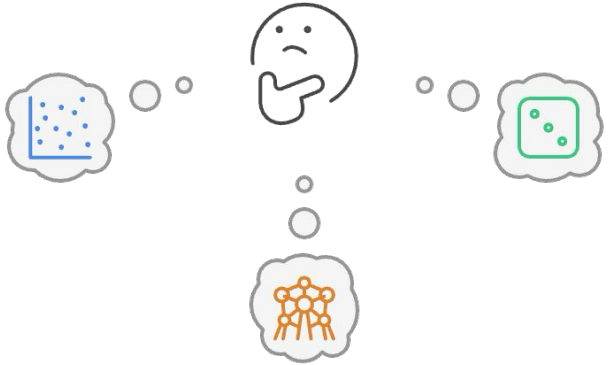
# Machine Learning Models

# Where Do ML-Based Models Fit in NLP?

**Which model type should be used for text analysis?**



**Vector-based models**

Convert text into numerical representations for further processing.

**Probabilistic models**

Use statistical probability to predict outcomes.

**ML-based models**

Learn patterns from data for classification and clustering.

# Vector-Based Models: Feature Representation

- Convert text into numerical form, but they do not predict!

Examples:

- **Bag of Words (BoW)** – Word occurrence
- **TF-IDF** – Word importance
- **Word Embeddings** – Word2Vec, GloVe

Used as input for ML models!

# Probabilistic Models: Learning with Probability

- Predict outcomes based on probability distributions

Examples:

- **Naïve Bayes**: P(Class | Words) for text classification
- **Markov Models**: Probabilities of word sequences (e.g., POS tagging)

Can be standalone or combined with ML!

# Machine Learning-Based Models: Learning from Data

- Learn patterns from labeled/unlabeled data
- Supervised Learning (e.g., Logistic Regression, SVM, Decision Trees)
- Unsupervised Learning (e.g., K-Means Clustering)
- Require vectorized input (from Vector-based models) for NLP tasks

# Applications

| Supervised | Unsupervised |
|---|---|
| Spam detection | Topic modeling |
| Sentiment analysis | Latent semantic analysis |
| | Text summarization |

# Spam Detection

# How Do We Detect Spam?

Three key steps:

1. Convert text into numerical features (Vectorization)
2. Train a model to classify spam vs. ham (ML-based model)
3. Evaluate model accuracy (Precision, Recall, F1-score)

We need a dataset of labeled emails to train the model!

# Step 1: Converting Emails into Features

Text must be converted into numerical format before ML can process it!

- Bag of Words (BoW) – Count how often words appear
- TF-IDF (Term Frequency-Inverse Document Frequency) – Weigh important words
- Word Embeddings (Word2Vec, GloVe) – Capture meaning and context

Example:

| Email Content | TF-IDF Features |
|---|---|
| "Win a free iPhone now!" | Free: 0.8, iPhone: 0.7, Win: 0.9 |
| "Meeting at 10 AM tomorrow" | Meeting: 0.6, Tomorrow: 0.7, 10 AM: 0.8 |

# Step 2: Training Machine Learning Models

We can use different ML models to classify emails:

| Model | Pros | Cons |
|---|---|---|
| Naïve Bayes | Fast & works well on small data | Assumes words are independent |
| Logistic Regression | Simple & interpretable | Struggles with complex text |
| Support Vector Machine (SVM) | Great for text classification | Slow with large datasets |
| Random Forest | Robust & handles noisy data | Requires more computation |

# Step 3: Evaluating Model Performance

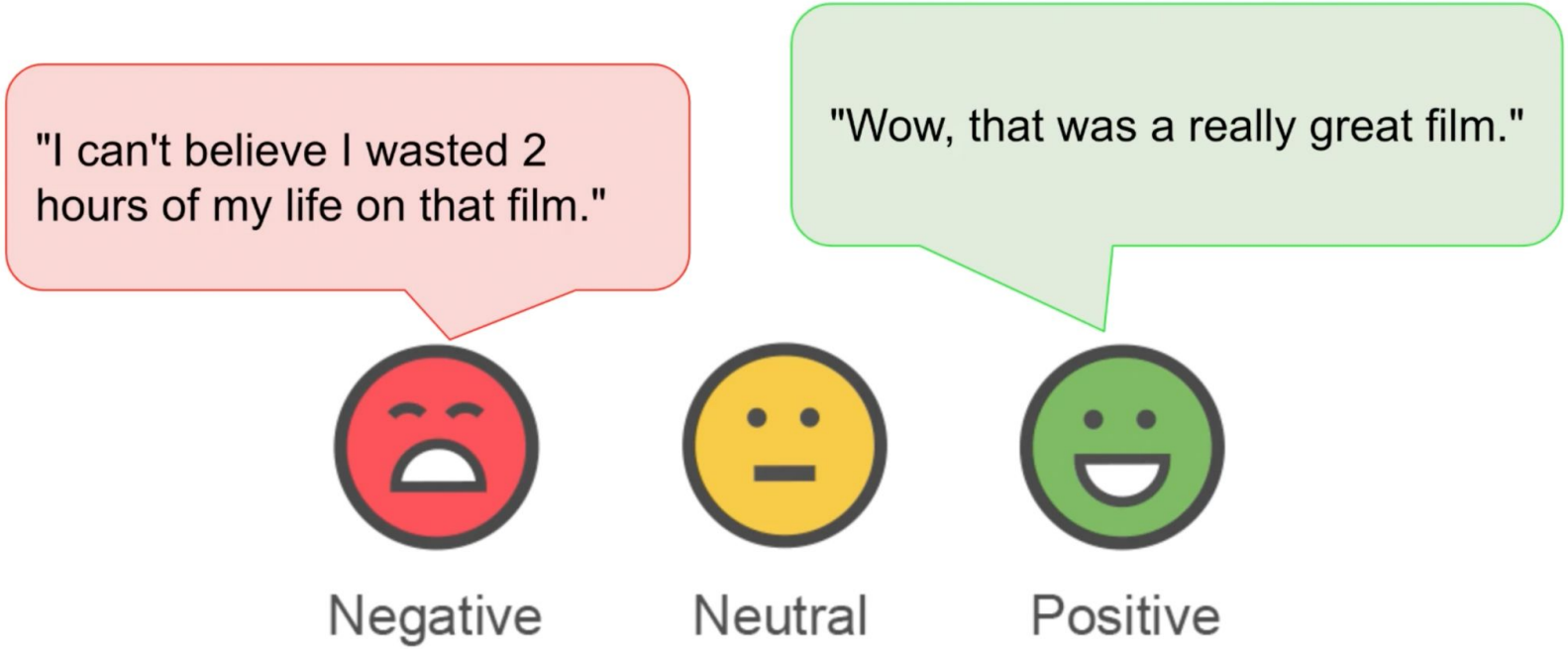
How do we measure if our model works well?

- Accuracy – Correct predictions / Total predictions
- Precision – How many predicted spam emails are actually spam?
- Recall – How many actual spam emails were detected?
- F1-Score – Balance between Precision & Recall
- Confusion Matrix

| | Predicted Spam | Predicted Ham |
|---|---|---|
| Actual Spam | ✅ 50 | ❌ 10 |
| Actual Ham | ❌ 5 | ✅ 100 |

# Sentiment Analysis

# Sentiment Analysis



# Classification or Regression

- Typically we consider sentiment analysis to be a classification task
- When we have multiple classes, we can also treat it as regression task
- Ex: 4.2 positive, -3.5 negetive

# Types of Sentiment Analysis

- **Binary Sentiment Analysis**: Classifies text as positive or negative.
- **Ternary Sentiment Analysis**: Categorizes text as positive, negative, or neutral.
- **Fine-Grained Sentiment Analysis**: Uses a scale (e.g., very positive, positive, neutral, negative, very negative).
- **Aspect-Based Sentiment Analysis**: Identifies sentiment towards specific aspects of a product or service.
  - For example, in a restaurant review like 'The food was amazing, but the service was slow,' aspect-based sentiment analysis would classify 'food' as positive and 'service' as negative.
- **Emotion Detection**: Recognizes emotions such as happiness, anger, sadness, etc.

# Approaches to Sentiment Analysis

- Lexicon-Based Approach:
  - Uses predefined word lists (sentiment lexicons) to determine sentiment.
    - Ex: 讚、喜歡、優秀，各得 +1 分；爛、討厭、雷，各得 -1 分
  - 除此之外，也會針對否定詞、程度副詞、標點符號等另行處理
    - 不好 → -1*1
    - 好吃 → +1; 超級好吃 → +1.5; 稍微好吃 → +0.5
    - 誇張 vs 誇張！！！
  - Example approach: SentiWordNet, AFINN.
- Machine Learning-Based Approach:
  - Uses algorithms such as Naive Bayes, Support Vector Machines (SVM), and Deep Learning.
  - Requires training data labeled with sentiment.

# Sentiment Analysis Tools & Libraries

- VADER
  - Suitable for social media sentiment analysis. It is rule-based and works well with short, informal text such as tweets.
  - Ex: LOL, WTF
- TextBlob
  - Simple API for NLP tasks, including sentiment analysis. Good for beginners and lightweight applications.
- BERT & Transformer Models
  - State-of-the-art NLP models that improve sentiment classification by understanding contextual meaning in text.

# Challenges in Sentiment Analysis

- **Sarcasm & Irony**
  - ex: 老師講的笑話真的太好笑了，不用開冷氣也不會熱
- **Context Understanding**
  - ex: 我沒事
  - Context 1 (陌生人) → postive or neutral
  - Context 2 (另一半) → negative !!!
- **Domain Dependency**
  - 劇情發展完全不可預測！ vs 系統運作狀態不可預測
- **Scope of Negation**
  - ex: 我不覺得這部電影的特效做得很好，但配樂還算可以。
  - “不”的影響範圍到底有多遠？

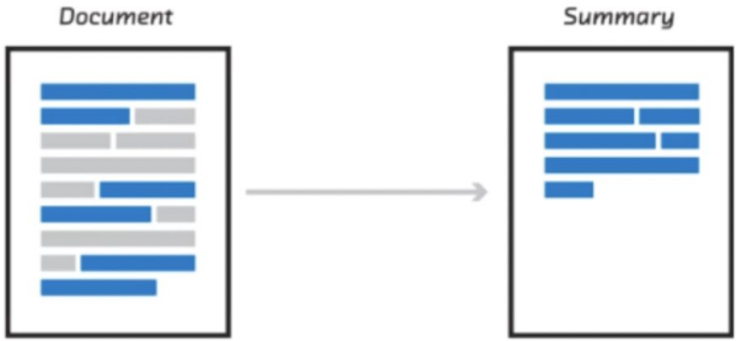
# Real-World Applications of Sentiment Analysis

- **Customer Feedback Analysis**: Understanding user satisfaction in reviews.
- **Social Media Monitoring**: Identifying public sentiment about brands and events.
- **Political Analysis**: Assessing public opinion on political issues.
- **Stock Market Predictions**: Analyzing sentiment from financial news and tweets.
- **Healthcare & Mental Health Monitoring**: Detecting signs of depression or stress through text analysis.

# Text Summarization

# Text Summarization

- We do this all the time
  - Scientific paper abstracts
  - Executive summaries in professional documents
  - …
- Summary also helps AI



# Why Text Summarization Helps AI

- “Summarization” is an aspect of “learning”
- E.g. In everyday conversations, we often paraphrase (i.e. summarize) what people tell you
- Summarization is a way for learning systems to demonstrate understanding of a concept
- Summarization can also be used for reducing the size of tokens passing to models (e.g. ChatGPT)

# 2 Types of Summarization

**Extractive Summarization**:

- Selects key sentences or phrases directly from the original text.
- Example: News summarization by picking the most relevant sentences.
- Tools: TextRank, LexRank, Sumy, RAKE.

**Abstractive Summarization**:

- Generates a summary using new words while retaining the original meaning.
- Example: Human-like rewriting of a paragraph in a more concise form.
- Tools: BERTSUM, Pegasus, T5, GPT-based models.

# Text Summarization with TF-IDF

- Split the document into sentences
- Score each sentence
- Rank each sentence by those scores
- Summary = top scoring sentences

# Sentence Splitting & TF-IDF

- Sentence tokenization (splitting document into sentences) can be done with NLTK (nltk.sent_tokenize(your_text))
- Built TF-IDF matrix, treating each sentence as if they were documents
- In previous example, the TF-IDF matrix was document x terms
- In this case, it will be sentences x terms

| | incubation | mean | of | period | risk | the | transmission |
|---|---|---|---|---|---|---|---|
| Sentence1 | high | medium | 0 | medium | 0 | 0 | 0 |
| Sentence2 | 0 | medium | 0 | medium | 0 | 0 | 0 |
| Sentence3 | 0 | 0 | 0 | 0 | medium | 0 | high |

# Scoring Each Sentence

- Score = Average(non-zero TF-IDF values)
- E.g. if row = [0, 1, 0, 0, 0, 2, 3, 0, 0, 0], then score = avg(1, 2, 3) = 2
- Why mean and not sum?
- The sum would be biased toward longer sentences

# What to Do with the Scores

- Idea: sort the scores, pick the sentences with the highest scores
- How? There are multiple options: choose what works best for you
- Simple: top N sentences (e.g. top 5, top 10)
- Also simple: top N words, top N characters (e.g. if limited by space)
- Top X% of sentences, top X% of words / characters

# TextRank

- Split the document into sentences
- Preprocessing
- Score each sentence
- Rank each sentence by those scores
- Summary = top scoring sentences

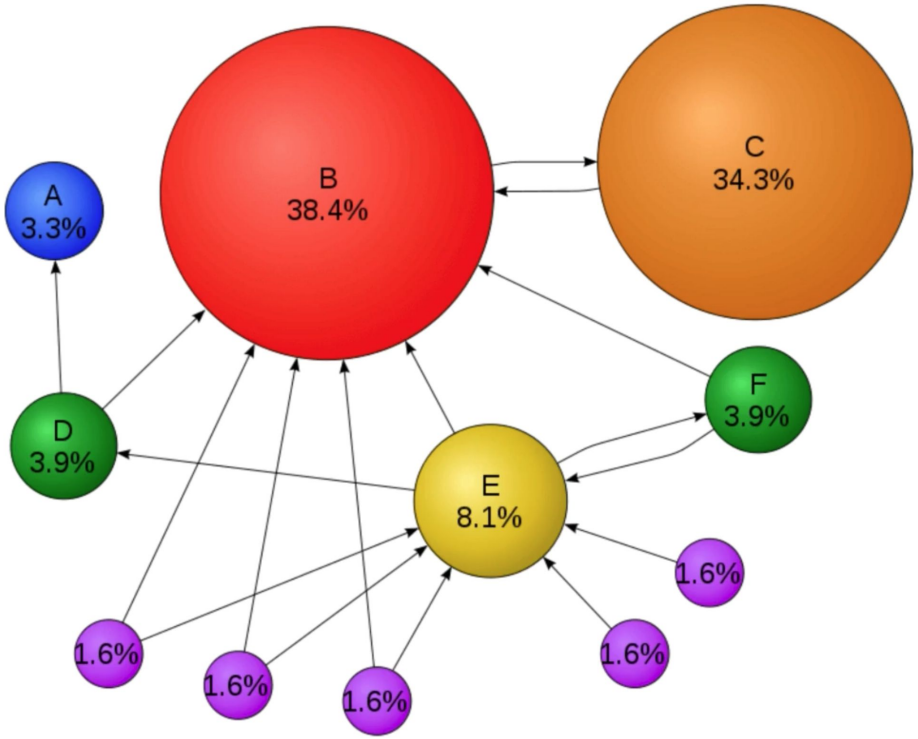
- TextRank is an alternative method of scoring each sentence
- All the other steps remain

# PageRank

- TextRank is based on PageRank, Google's state-of-the-art search ranking method ~20 years ago which beat all competitors and led to Google becoming one of the largest tech companies in the world
- The Internet is made up of web pages, each of which can potentially be returned as a search result
- We would like to compute a score for each webpage

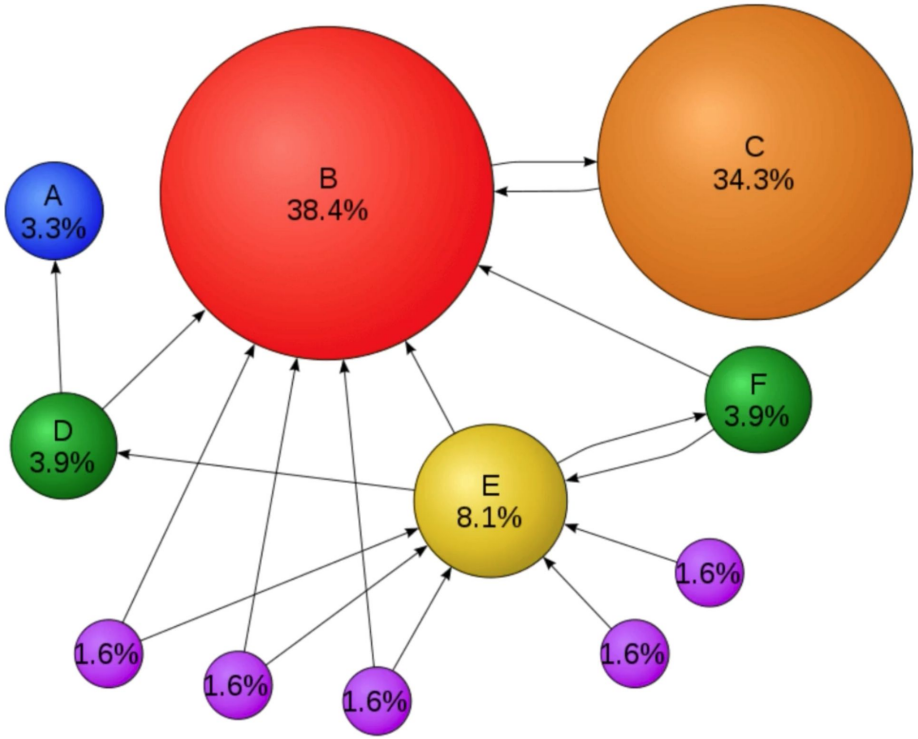
# The Secret to PageRank

- The secret to scoring each web page is the random walk
- Starting from an arbitrary page, randomly select a link, go to that page, repeat forever…
- After a “long time”, the probability of landing on any particular page is constant (convergence)
- Doesn’t matter where you start!



# PageRank Intuition

- A web page with more incoming links is more popular, and will have a higher chance of being landed on
- A web page few or no links is not popular, so the probability of going there is smaller
- These probabilities are the PageRank scores



# Applying PageRank to TextRank

- There are two TextRank methods: symmetric and asymmetric
- Symmetric is suitable for sentence extraction as we evaluate the similarity between two sentences, there is no direction
- Asymmetric is suitable for keyword extraction because the word order matters
  - Ex: “人工”後面常常接“智慧”, 但“智慧”後面可能常接的是“手錶”、“財產”

# Symmetric TextRank

The symmetric TextRank algorithm:

$$S(V_i) = (1-d) + d \times \sum_{V_j \in N(V_i)} \frac{w_{ij}}{\sum_{V_k \in N(V_j)} w_{jk}} S(V_j)$$

- $S(V_i)$: 句子 i 目前的重要性分數 (Score)
- d: 阻尼係數，通常設為 0.85。代表15% 的機率會隨機跳到其他句子，避免分數流動卡死在某個封閉的小圈圈裡
- $N(V_i)$: 與句子 i 相連的所有鄰居節點的集合
- $w_{ij}$: 句子 i 與句子 j 之間的相似度權重
- $\Sigma V_k \in N(V_j)\ w_{jk}$: 這是句子 j 跟它所有鄰居連線的權重總和。把它放在分母，是為了計算出一個分配比例

# Symmetric TextRank

假設我們有三個句子 (斷詞版)

- S1: 機器 學習 很 有趣
- S2: 機器 學習 很 困難
- S3: 深度 學習 解決 困難

假設所有句子的起始分數都是 1

Step 1: 計算句子間的相似度

- Can use cooccurence counting or cosine similarity
- S1 與 S2: 共有「機器、學習、很」3 個詞 → $w_{12} = w_{21} = 3$
- S1 與 S3: 共有「學習」1 個詞 → $w_{13} = w_{31} = 1$
- S2 與 S3: 共有「學習、困難」2 個詞 → $w_{23} = w_{32} = 2$

# Symmetric TextRank

假設我們有三個句子 (斷詞版)

- S1: 機器 學習 很 有趣
- S2: 機器 學習 很 困難
- S3: 深度 學習 解決 困難

假設所有句子的起始分數都是 1

- w12 = w21 = 3
- w13 = w31 = 1
- w23 = w32 = 2

Step 2: 計算分母

- S1 的總權重 ($W_1$) = $w_{12} + w_{13} = 3 + 1 = 4$
- S2 的總權重 ($W_2$) = $w_{21} + w_{23} = 3 + 2 = 5$
- S3 的總權重 ($W_3$) = $w_{31} + w_{32} = 1 + 2 = 3$

# Symmetric TextRank

假設我們有三個句子 (斷詞版)

- S1: 機器 學習 很 有趣
- S2: 機器 學習 很 困難
- S3: 深度 學習 解決 困難

假設所有句子的起始分數都是 1

- w12 = w21 = 3
- w13 = w31 = 1
- w23 = w32 = 2
- $W_1 = 4$
- $W_2 = 5$
- $W_3 = 3$

Step 3: 迭代開始

- Update S1
  - 從 S2 那邊可以分到 ⅗ 的比例
  - 從 S3 那邊可以分到 ⅓ 的比例
  - $S(S_1) = (1 - 0.85) + 0.85 * (⅗ * 1 + ⅓ * 1) = 0.943$

$$S(V_i) = (1-d) + d \times \sum \left( \frac{\text{彼此的權重}}{\text{對方的總權重}} \times \text{對方的分數} \right)$$

# Symmetric TextRank

假設我們有三個句子 (斷詞版)

- S1: 機器 學習 很 有趣
- S2: 機器 學習 很 困難
- S3: 深度 學習 解決 困難

假設所有句子的起始分數都是 1

- w12 = w21 = 3
- w13 = w31 = 1
- w23 = w32 = 2
- $W_1 = 4$
- $W_2 = 5$
- $W_3 = 3$

Step 3: 迭代開始

- Update S2
  - 從 S1 那邊可以分到 ¾ 的比例
  - 從 S3 那邊可以分到 ⅔ 的比例
  - $S(S_2) = (1 - 0.85) + 0.85 * (\frac{3}{4} * 1 + \frac{2}{3} * 1) = 1.354$

$$S(V_i) = (1-d) + d \times \sum \left( \frac{\text{彼此的權重}}{\text{對方的總權重}} \times \text{對方的分數} \right)$$

# Symmetric TextRank

假設我們有三個句子 (斷詞版)

- S1: 機器 學習 很 有趣
- S2: 機器 學習 很 困難
- S3: 深度 學習 解決 困難

假設所有句子的起始分數都是 1

- w12 = w21 = 3
- w13 = w31 = 1
- w23 = w32 = 2
- $W_1 = 4$
- $W_2 = 5$
- $W_3 = 3$

Step 3: 迭代開始

- Update S3
  - 從 S1 那邊可以分到 ¼ 的比例
  - 從 S2 那邊可以分到 ⅖ 的比例
  - $S(S_3) = (1 - 0.85) + 0.85 * (¼ * 1 + ⅖ * 1) = 0.703$

$$S(V_i) = (1-d) + d \times \sum \left( \frac{\text{彼此的權重}}{\text{對方的總權重}} \times \text{對方的分數} \right)$$

# Symmetric TextRank

假設我們有三個句子 (斷詞版)

- S1: 機器 學習 很 有趣
- S2: 機器 學習 很 困難
- S3: 深度 學習 解決 困難

假設所有句子的起始分數都是 1

- w12 = w21 = 3
- w13 = w31 = 1
- w23 = w32 = 2
- $W_1 = 4$
- $W_2 = 5$
- $W_3 = 3$

## Step 3: 迭代開始

- Update S3
  - 從 S1 那邊可以分到 ¼ 的比例
  - 從 S2 那邊可以分到 ⅘ 的比例
  - $S(S_3) = (1 - 0.85) + 0.85 * (¼ * 1 + ⅘ * 1) = 0.703$

第一輪結束後:
S1: 1→0.94
S2: 1→**1.35**
S3: 1→0.7
Repeat Step3 直到收斂

$$S(V_i) = (1-d) + d \times \sum \left( \frac{\text{彼此的權重}}{\text{對方的總權重}} \times \text{對方的分數} \right)$$

# Evaluating Summarization Quality

How to Assess Summarization?

- Evaluating summarization requires both automatic and human evaluation methods.
- Key criteria: Relevance, Fluency, Coherence, Conciseness, and Faithfulness.

# Automatic Evaluation – ROUGE

- Recall-Oriented Understudy for Gisting Evaluation
- In summarization, omission is worse than redundancy
- ROUGE-N (N-gram matching) & ROUGE-L (Longest Common Subsequence)

Reference: 小明 喜歡 吃 蘋果

Generated: 小明 非常 喜歡 吃 紅色 的 蘋果

ROUGE-1 (是否有關鍵字):

- Overlap: 小明、喜歡、吃、蘋果
- Recall = 4/4 = 100%
- Precision = 4/7 = 57.1%
- F1-score = 72.7%

# Automatic Evaluation – ROUGE

- Recall-Oriented Understudy for Gisting Evaluation
- In summarization, omission is worse than redundancy
- ROUGE-N (N-gram matching) & ROUGE-L (Longest Common Subsequence)

Reference: 小明 喜歡 吃 蘋果

Generated: 小明 非常 喜歡 吃 紅色 的 蘋果

ROUGE-2 (是否流暢):

- Overlap: [喜歡 吃]
- Recall = 1/3 = 33.3%
- Precision = 1/6 = 16.7%
- F1-score = 22.2%

# Automatic Evaluation – ROUGE

- Recall-Oriented Understudy for Gisting Evaluation
- In summarization, omission is worse than redundancy
- ROUGE-N (N-gram matching) & ROUGE-L (Longest Common Subsequence)

Reference: 小明 喜歡 吃 蘋果

Generated: 小明 非常 喜歡 吃 紅色 的 蘋果

ROUGE-L (句子架構):

- LCS: 小明 → 喜歡 → 吃 → 蘋果 (長度為 4)
- Recall = len(LCS)/len(Ref) = 4/4 = 100%
- Precision = len(LCS)/len(Gen) = 4/7 =57.1%
- F1-score = 72.7%

# Automatic Metrics

| Metric | Description | Strengths | Limitations |
|---|---|---|---|
| ROUGE | Measures overlap of n-grams between generated and reference summaries | Fast, widely used, good for extractive summaries | Favors surface similarity, ignores meaning |
| BLEU | Counts n-gram overlaps | Good for short summaries | Penalizes paraphrasing, not ideal for summarization |
| METEOR | Uses stemming & synonym matching | More robust than BLEU | Computationally complex |
| BERTScore | Uses deep learning to measure semantic similarity | Captures meaning better than ROUGE/BLEU | Requires pre-trained models |

# Human Evaluation

| Aspect | Description | Example Evaluation Criteria |
|---|---|---|
| Relevance | Captures key information from source | Are essential details included? |
| Fluency | Grammatically correct & readable | Does the summary read smoothly? |
| Coherence | Logical sentence connections | Does the summary flow naturally? |
| Conciseness | Brief yet informative | Does it remove unnecessary words? |
| Faithfulness | Accurately reflects source content | Is it free from misinformation? |

# Challenges in Text Summarization

- **Context Understanding**: Struggling to capture deep semantic meaning, especially in long documents.
- **Coherence**: Summaries assembled from extracted sentences may lack natural logical flow.
- **Information Loss**: High risk of omitting crucial details from the source text.
- **Bias & Hallucination**: AI-generated summaries might introduce misinformation.